```
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
            // Process the read bytes
            System.out.println("Read " + bytesRead + " bytes.");
        }

        fis.close();
    }
}
```

Output

FileNotFoundException occurred: non-existent-file.txt (No such file or directory)

Introduction to design patterns in Java : Singleton and Adaptor.

What are Design Patterns?

- Predefined **solutions** to common design problems.
- Help write **cleaner, reusable, and maintainable code**.
- Categories:
    - **Creational** (object creation) → e.g., *Singleton*
    - **Structural** (class/object composition) → e.g., *Adapter*
    - **Behavioral** (communication between objects) → e.g., *Observer*

1. Singleton Pattern (Creational)

🎯 **Purpose:**

Ensure a **class has only one instance** and provide a **global point of access** to it.

📌 **Key Points:**

- Constructor is **private**
- One **static instance** of the class
- Access via a **public static method**

Example:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {
        System.out.println("Singleton instance created");
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();  // Lazy initialization
```

```
        }
        return instance;
    }
}
```

✅ **Usage:**

```
public class Main {
    public static void main(String[] args) {
        Singleton obj1 = Singleton.getInstance();
        Singleton obj2 = Singleton.getInstance();

        System.out.println(obj1 == obj2);  // true
    }
}
```

💡 **Where to Use:**

- Logger
- Configuration manager
- DB connection pool

2. Adapter Pattern (Structural)

🎯 **Purpose:**

Convert the **interface of one class** into another expected by the client.
Think: "I have a plug that doesn't fit – use an adapter."

📌 **Key Points:**

- Used to **bridge incompatible interfaces**
- Acts like a **translator**

✅ **Example Scenario:**

You have a media player that only plays mp3. But you want to play vlc files too using an existing VLC player class.

✅ **Code Example:**

Step 1: Target interface

```
interface MediaPlayer {
    void play(String audioType, String fileName);
}
```

Step 2: Adaptee (existing class)

```
class AdvancedMediaPlayer {
    void playVLC(String fileName) {
        System.out.println("Playing VLC: " + fileName);
    }
}
```

Step 3: Adapter class

```
class MediaAdapter implements MediaPlayer {
    AdvancedMediaPlayer player = new AdvancedMediaPlayer();

    public void play(String audioType, String fileName) {
        if (audioType.equalsIgnoreCase("vlc")) {
            player.playVLC(fileName);
        } else {
            System.out.println("Format not supported");
        }
    }
}
```

Step 4: Client

```
class AudioPlayer implements MediaPlayer {
    MediaAdapter adapter;

    public void play(String audioType, String fileName) {
        if (audioType.equalsIgnoreCase("mp3")) {
            System.out.println("Playing MP3: " + fileName);
        } else {
            adapter = new MediaAdapter();
            adapter.play(audioType, fileName);
        }
    }
}
```

✅ **Usage:**

```
public class Main {
    public static void main(String[] args) {
        AudioPlayer player = new AudioPlayer();
        player.play("mp3", "hello.mp3");
        player.play("vlc", "movie.vlc");
    }
}
```

💡 **Where to Use:**

- Connecting legacy code with modern systems
- File format converters
- Device drivers